

INFORME TÉCNICO MLOPS

Servicio end-to-end de predicción de fuga de clientes

Asignatura: MLOps · MDS UAI

Repositorio: github.com/LeoSaraviaS/churn-mlops

Modelo: Regresión logística

Fecha: 16 de agosto de 2026

Resumen: El proyecto implementa un servicio reproducible para estimar la probabilidad de fuga de clientes bancarios. La solución integra entrenamiento, evaluación, API REST, pruebas, Docker, gates CI/CD, smoke test sobre el contenedor, publicación en GHCR y una instancia pública operativa en Google Cloud Run

1. Problema y objetivo

El objetivo es predecir la variable Exited, donde 1 representa fuga y 0 permanencia. La predicción entrega una clase, una probabilidad y un nivel de riesgo, con el fin de priorizar acciones de retención. El problema es de clasificación binaria y se evalúa sobre observaciones no utilizadas durante el entrenamiento.

Datos y preparación

El dataset Churn_Modelling.csv contiene 10.000 registros y 14 columnas. No presenta valores faltantes ni filas duplicadas. La clase está desbalanceada: 7.963 clientes permanecen (79,63 %) y 2.037 abandonan (20,37 %). Por ello no se utiliza accuracy como única medida de desempeño.

Elemento	Decisión aplicada
Variable objetivo	Exited: 1 = fuga; 0 = permanencia
Identificadores excluidos	RowNumber, CustomerId y Surname
Separación	80 % entrenamiento y 20 % prueba; random_state=42; stratify=y
Fuente	Dataset y notebook públicos de Kaggle, citados en el README

2. Decisiones de diseño

La arquitectura separa entrenamiento e inferencia. El modelo se entrena fuera del servicio, se guarda como artefacto y FastAPI lo carga una sola vez al iniciar. Esta separación evita reentrenamientos durante una petición y permite reproducir la misma transformación en entrenamiento y producción.

Etapas	Componente	Responsabilidad
Datos	data/	Dataset original versionado para entrenamiento reproducible
Entrenamiento	training/	Validación, limpieza, split, pipeline, métricas y gate ROC-AUC
Artefactos	models/	Pipeline serializado y metadata con versiones, fecha y resultados

Etapa	Componente	Responsabilidad
Servicio	app/	Contratos Pydantic, carga del modelo, predicción y métricas
Operación	Docker + Actions	Entorno portable y automatización de controles de calidad

Modelo y prevención de inconsistencias

Se utiliza LogisticRegression con `class_weight="balanced"`, `max_iter=1000` y semilla fija. El pipeline deriva AgeGroup, codifica Geography, Gender y AgeGroup con OneHotEncoder y estandariza las variables numéricas. Las transformaciones se ajustan solamente con el conjunto de entrenamiento y quedan integradas dentro de `model.joblib`, reduciendo el riesgo de fuga de información y de diferencias entre entrenamiento e inferencia.

Resultados sobre datos no vistos

Métrica	Resultado
Accuracy	0,7115
Precision — clase fuga	0,3861
Recall — clase fuga	0,7076
F1-score — clase fuga	0,4996
ROC-AUC	0,7770
Matriz de confusión	TN=1.135 · FP=458 · FN=119 · TP=288

Interpretación: el modelo recupera aproximadamente el 71 % de los clientes que efectivamente abandonan, a costa de una precisión menor y un número relevante de falsos positivos. El ROC-AUC supera el gate mínimo de 0,75, pero el umbral operativo debe ajustarse según el costo de contactar clientes y el costo de perderlos.

3. Implementación MLOps

El servicio usa FastAPI y contratos Pydantic con validación de categorías y rangos. El contenedor se basa en `python:3.12-slim`, instala dependencias fijadas, ejecuta con usuario `no-root` y contiene HEALTHCHECK. `docker-compose.yml` permite reproducir el servicio localmente.

Estado de la API

Endpoint	Estado	Observación
GET /health	Operativo	Verificado en Cloud Run, modelo cargado
POST /predict	Operativo	Verificado en Cloud Run con respuesta 200

Endpoint	Estado	Observación
GET /metrics	Operativo	Verificado en Cloud Run, métricas Prometheus disponibles
POST /predict/batch	Operativo	Verificado en Cloud Run con resumen agregado
GET /model/schema	Operativo	Verificado en Cloud Run, contrato y versión disponibles

Pruebas y CI/CD

El repositorio contiene 32 pruebas unitarias y de integración, además de Ruff. El workflow se ejecuta en push y PR hacia dev y main. Las ejecuciones más recientes del commit e9570dd completaron correctamente lint, test, train-and-gate, docker build y smoke-test, deploy y publish GHCR. La prueba de humo construye y levanta el contenedor, valida /health y prueba /predict con payload válido y con un caso inválido que debe responder 422. Cuando la imagen se ejecuta correctamente con el smoke test, se almacena oficialmente en Google Artefacts Registry.

Versionado, publicación y despliegue

La prueba v0.1.0 completó los gates y publicó la imagen en GHCR como ghcr.io/leosaravias/churn-api:v0.1.0 y latest. El commit e9570dd dejó la publicación condicionada al smoke test. La instancia pública de Cloud Run responde en <https://churn-api-373252903708.southamerica-west1.run.app>; el 16 de agosto se verificaron /health, /predict, /predict/batch, /model/schema y /metrics. La entrega final aún requiere integrar dev en main y crear el tag v1.0.0 sobre el commit definitivo.

4. Limitaciones y trabajo pendiente

Limitaciones	Impacto y tratamiento propuesto
Desbalance y precisión	Baja precisión y falsos positivos. Ajustar el umbral según costos de negocio.
Datos históricos	Sin validación temporal ni drift. Monitorear distribuciones y desempeño.

Equipo trabajo

Equipo	Tareas
Leonor Saravia /LeoSaraviaS	Repositorio y arquitectura base
Valentina Ariztía /valentinaprogramando	PR #7 integrado; smoke test Docker, CI en dev/main, URL Cloud Run, README y publicación GHCR posterior al smoke.
Jonathan Machuca /jomanss	PR #3 y #5 integrados: batch, README, .env y limitaciones. PR #6 integrado
Hemersson Gutiérrez /Hemersson7	/model/schema, reproducibilidad, curva ROC, revisiones y merges de los PR #3, #4 y #5.

Equipo	Tareas
Alejandra Véliz /alejandraveliz-crypto	PR #4 integrado: GHCR v1.0.0 PR #8 integrado

Uso de inteligencia artificial

El equipo declaró el uso de Claude como apoyo conceptual, de redacción y de programación durante el desarrollo. Para revisar la pauta y el repositorio, estructurar el informe y apoyar su redacción se utilizó además ChatGPT/Codex. El equipo conserva la responsabilidad sobre las decisiones, la ejecución, la validación y la entrega.

Fuentes

Dataset y notebook: kaggle.com/datasets/anandshaw2001/customer-churn-dataset · kaggle.com/code/ih8asham/customer-churn-dataset

Rpositorio y pauta: github.com/LeoSaraviaS/churn-mlops · Actions · GHCR · Cloud Run · Pauta UAI.